Case Study: Hospital Management System

Scenario

A **hospital** maintains patient treatment records in a MongoDB collection called **`patients`**.

Each patient record contains:

- patientName
- doctor
- department
- age
- treatmentCost
- daysAdmitted

👉 The hospital wants to:

- Analyze treatment costs
- Track doctors and departments
- Generate reports using queries

Insert Many Documents (MongoDB)

```
db.patients.insertMany([
{ patientName: "Arun", doctor: "Dr. Ravi", department: "Cardiology", age: 45, treatmentCost: 20000, daysAdmitted: 5 },
{ patientName: "Meena", doctor: "Dr. Kumar", department: "Neurology", age: 50, treatmentCost: 30000, daysAdmitted: 7 },
{ patientName: "Raj", doctor: "Dr. Ravi", department: "Cardiology", age: 60, treatmentCost: 15000, daysAdmitted: 3 },
{ patientName: "Divya", doctor: "Dr. Mehta", department: "Orthopedics", age: 35, treatmentCost: 10000, daysAdmitted: 2 },
{ patientName: "Kiran", doctor: "Dr. Kumar", department: "Neurology", age: 40, treatmentCost: 25000, daysAdmitted: 6 },
{ patientName: "Sneha", doctor: "Dr. Mehta", department: "Orthopedics", age: 30, treatmentCost: 8000, daysAdmitted: 2 },
```

{ patientName: "Vijay", doctor: "Dr. Ravi", department: "Cardiology", age: 55, treatmentCost: 22000, daysAdmitted: 4 },
{ patientName: "Priya", doctor: "Dr. Kumar", department: "Neurology", age: 48, treatmentCost: 27000, daysAdmitted: 5 },
{ patientName: "Anil", doctor: "Dr. Mehta", department: "Orthopedics", age: 52, treatmentCost: 12000, daysAdmitted: 3 },
{ patientName: "Riya", doctor: "Dr. Ravi", department: "Cardiology", age: 29, treatmentCost: 18000, daysAdmitted: 4 }
])

1. Find all patients whose treatment cost is greater than 20,000 and display only their name and department.

Explanation

Uses `$match` to filter documents based on `treatmentCost`
Uses `$project` to include only `patientName` and `department` fields

2. List all patients sorted by treatment cost in descending order.

Explanation

Uses `$sort` to arrange documents by `treatmentCost` in descending order

3. Display the first 5 patient records.

Explanation

Uses `$limit` to restrict the output to the first 5 documents

4. Skip first 3 patients and display the remaining records.

Explanation

Uses `$skip` to ignore the first 3 documents

5. Find patients from the Cardiology department and display only their names.

Explanation

`$match` filters documents where `department = "Cardiology"`

`$project` selects only the `patientName` field

6. Find total treatment cost per department.

Explanation

Uses `$group` to group documents by `department`
Uses `$sum` to calculate total cost within each group

7. Find average treatment cost per doctor and display results in ascending order.

Explanation

`$group` groups documents by `doctor`
`$avg` calculates average treatment cost
`$sort` arranges results in ascending order

8. Find departments where total treatment cost is greater than 50,000.

Explanation

`$group` computes total cost per department
A second `$match` filters grouped results (like HAVING)

9. Find number of patients in each department and sort the result in descending order.

Explanation

`$group` counts documents using `$sum: 1`
`$sort` orders results by count

10. Find top 3 doctors with highest total treatment cost.

Explanation

`$group` calculates total cost per doctor
`$sort` arranges in descending order
`$limit` selects top 3 documents

11. Find patients whose age is greater than 40, group them by department, and count number of patients.

Explanation

`$match` filters patients by age
`$group` groups by department
`$sum: 1` counts patients

12. Find average treatment cost per department, display only department and average cost, and sort descending.

Explanation

`$group` calculates average cost per department
`$project` reshapes output fields
`$sort` orders results

13. Find departments having more than 2 patients admitted.

Explanation

`$group` counts number of patients
Second `$match` filters departments with count > 2

14. Display top 3 departments based on highest total treatment cost after skipping the first one.

Explanation

`$group` calculates total cost per department
`$sort` sorts in descending order
`$skip` ignores the first document
`$limit` selects next 3 documents

15. Find doctors treating patients with cost greater than 10,000, calculate total cost, and show only top 2 doctors.

Explanation

`$match` filters high-cost treatments
`$group` sums cost per doctor
`$sort` orders by total cost

`$limit` selects top 2 doctors